
CACHE-AWARE ROUTING FOR MULTI-NODE LLM INFERENCE

Aman Pandey Sanjeev Kumar¹ Guna Avula¹ Aryan Khanolkar¹

ABSTRACT

Distributed LLM inference systems commonly reuse key-value (KV) caches to accelerate decoding, but this reuse is typically limited to a single worker. In multi-node deployments, stateless routers prioritize load balancing and remain unaware of prefix locality, leading to redundant prefill computation and increased time-to-first-token (TTFT), particularly for workloads with shared system prompts, multi-turn chat, and retrieval-augmented generation (RAG). In this work, we study cache-aware routing policies for multi-node LLM serving using a vLLM-inspired event-driven latency simulator. In order to realistically simulate vLLM instances, our workers model paged KV caches, prefix-based reuse, eviction under capacity pressure, and continuous batching under a token budget. The router maintains a lightweight, optimistic view of each worker’s cached prefixes. We implement four routing strategies, including a queue-and-prefill cost-aware policy that uses an online estimate of per-token decode time to compare queue wait against prefill savings. Across burst and streaming workloads with varying degrees of prefix locality, we find that greedy cache-maximizing routing can significantly increase tail latency due to worker hotspots, despite achieving high cache hit rates. In contrast, our queue-and-prefill cost-aware policy yields more stable TTFT and end-to-end latency while retaining much of the cache reuse benefit. These results highlight that effective distributed LLM routing must balance cache locality and load to avoid degrading user-visible latency.

1 INTRODUCTION

Large Language Models (LLMs) are autoregressive, generating one token at a time. To make inference feasible, modern engines cache attention keys and values (KV cache) so decoding reuses prior states instead of recomputing the entire prefix (Kwon et al., 2023). With rising demand, LLM serving is deployed across multiple workers, where a front-end router decides which worker handles each request.

However, standard routing policies (e.g., round-robin, least-loaded) are cache-blind: they do not account for which worker already holds the KV cache for a request’s prefix. Under workloads with shared prefixes—such as system prompts, multi-turn chat, and Retrieval-Augmented Generation (RAG)—this blindness is costly. It induces redundant prefill computation, increases Time-To-First-Token (TTFT), and wastes valuable accelerator memory bandwidth.

In this work, we address the challenge of bringing cache awareness to distributed inference without introducing complex orchestration overhead. We propose a lightweight, vLLM-aware router that explicitly models the trade-off between the computational cost of prefill (cache miss) and the latency cost of queueing (cache hit on a busy worker).

¹Purdue University, West Lafayette, IN, USA. Correspondence to: Aman Pandey <sanjeevk@purdue.edu>.

Our main contributions are as follows:

- **Queue-and-Prefill Cost Model:** We formulate a routing objective that explicitly prices queue wait time against prefill recomputation time, allowing the system to dynamically switch between load balancing and cache locality based on estimated latency.
- **Lightweight State Management:** We introduce a “piggybacking” protocol that maintains an eventually consistent global prefix trie without heavy synchronization or central locking.
- **High-Fidelity Latency Simulation:** We evaluate our approach on a calibrated, event-driven latency simulator against real-world RAG workloads, demonstrating that our cost-aware policy prevents the tail latency degradation seen in naive greedy approaches.

2 PROBLEM FORMULATION

In a distributed multi-worker deployment, prefix reuse becomes topology dependent. To benefit from cached prefixes, a request must be routed to the specific worker that already holds the relevant KV blocks in its GPU memory. This creates a fundamental tension between data locality and load balancing.

A naive cache-aware strategy (Greedy Routing) routes every request to the worker with the maximum prefix match.

While this maximizes the cache hit rate, it introduces a critical failure mode known as herding. In real-world traces, specific prefixes (e.g., a popular RAG context or a shared system prompt) often receive bursts of traffic. A greedy router will saturate the single worker holding that prefix, creating a hotspot. As the queue depth on that worker grows, the wait time eventually exceeds the time it would have taken to simply recompute the prefix on an idle worker. Therefore, this project seeks to answer the following research questions:

1. Does maximizing cache hits (Greedy Routing) always reduce latency, or does it introduce resource contention under bursty, high-locality workloads in practice?
2. How can we quantify the exact latency trade-off between a cache hit on a busy worker (queue wait) versus a cache miss on an idle worker (recomputation)?
3. Can a single cost-based policy effectively adapt to varying traffic patterns, maintaining cache benefits during steady-state streaming while avoiding tail-latency degradation during bursts?

3 RELATED WORK

To understand the necessity for sophisticated routing in LLM serving, one must first deconstruct the execution characteristics of Transformer-based models.

3.1 Characteristics of LLM Inference

As detailed in foundational analyses of inference systems (Patel et al., 2024; Zhong et al., 2024), the first phase, which processes the user’s input prompt, is known as the prefill phase. Because the attention mechanism can process all input tokens in parallel, this phase is characteristically compute-bound. The arithmetic intensity, defined as the ratio of floating-point operations to bytes of memory accessed, is high, allowing modern accelerators to achieve high Model Flops Utilization (MFU).

The second phase, decode, generates tokens autoregressively, one at a time. Each step is dependent on the previous one, forcing a serial execution path. Crucially, generating a single token requires loading the entire model weights and the specific request’s KV cache from High Bandwidth Memory (HBM) to on-chip SRAM. Because only a small amount of computation is performed per byte loaded, this phase is severely memory-bandwidth bound (Dao et al., 2022).

In a distributed setting, a cache miss is catastrophic. It forces the system to revert from the computationally lightweight, memory-bound decode phase back to the heavy, compute-bound prefill phase to recompute the KV states. This recomputation stalls the pipeline, degrading the inter-token la-

tency for all concurrent requests sharing that GPU. Without coordination, local schedulers cannot resolve cluster-wide contention or exploit multi-node memory pools efficiently.

3.2 Disaggregation Strategies

One approach to address the fundamental difference in needs between prefill and decode phases is through disaggregated architectures, which separate the phases physically or logically.

1. **Physical Disaggregation:** Systems such as Splitwise (Patel et al., 2024) and DistServe (Zhong et al., 2024) propose physically separating the cluster into distinct prefill instances and decode instances. This allows for specialized parallelism strategies; for instance, DistServe uses high tensor parallelism for prefill latency and pipeline parallelism for decode throughput. However, this approach introduces significant network overhead. Transferring gigabytes of KV states between nodes requires expensive, high-bandwidth interconnects (e.g., InfiniBand) often unavailable in commodity cloud environments.
2. **Logical Disaggregation:** To avoid network bottlenecks, systems like DuetServe (Gao et al., 2025) and Nexus (Shi et al., 2025) implement logical disaggregation within a single GPU. They employ granular resource partitioning, isolating specific Streaming Multiprocessors (SMs) for prefill tasks to prevent them from starving decode tasks. While logical disaggregation eliminates state transfer costs, it requires complex, non-portable kernel modifications that are difficult to integrate into standard serving pipelines.

While disaggregation effectively isolates the computational characteristics of prefill and decode, it imposes strict hardware and orchestration requirements. Our work explores a more accessible alternative: optimizing standard homogeneous workers by shifting the complexity to a cache-aware routing layer.

3.3 State Management and Caching

To support this routing paradigm, the KV cache must be treated as a persistent asset rather than a transient buffer. SGLang (Zheng et al., 2024) revolutionized this approach by structuring the KV cache as a Radix Tree (prefix tree) instead of a linear buffer, enabling multiple requests with shared prefixes (e.g., system prompts) to map to the same physical memory blocks.

Recent architectures like Mooncake (Qin et al., 2024) and LMCache (Cheng et al., 2025) extend this concept to tiered storage, pooling memory across HBM, CPU RAM, and

NVMe SSDs via RDMA. While these systems allow for effectively infinite context caching, fetching data across the network—even via zero-copy RDMA—incurrs significantly higher latency than local HBM access. Consequently, minimizing latency in non-disaggregated clusters fundamentally relies on routing requests to the specific node where data is already locally resident.

3.4 Distributed Routing Logic

The introduction of persistent state complicates load balancing. A traditional router (e.g., Round-Robin or Least-Loaded) is typically stateless. Frameworks like the Power-of-Two-Choices (PoTC) (Mitzenmacher, 2001) suggest probing just random servers and selecting least loaded one yields exponentially better load balancing. However, PoTC assumes homogeneous service rates. In LLM inference, the service time of a request is highly dependent on the destination node’s cache state - a cache hit is orders of magnitude faster than a cache miss.

Existing routing solutions fall into two categories, neither of which fully addresses the trade-off between locality and load:

1. **Monolithic Runtimes:** SGLang (Zheng et al., 2024) integrates a sophisticated router that balances cache hits against queue length. However, this router is bound to the SGLang runtime and its specific domain-specific language (DSL). It cannot be decoupled to serve as a frontend for standard vLLM deployments, forcing users to migrate their entire inference stack to adopt SGLang’s ecosystem. Similarly, llm-d (llm-d Contributors, 2025) offers a distributed inference framework, but it is architected as a heavy, Kubernetes-native solution. This introduces significant infrastructure complexity, requiring a full container orchestration platform even for simple, small-scale clusters. It lacks the flexibility to operate as a lightweight router or standalone proxy.
2. **Adapter-Specific Routing:** Systems like LoRAServe (Sheng et al., 2023) and Punica (Chen et al., 2023) address a similar locality problem: routing requests to workers that already hold specific Low-Rank Adapters (LoRA) in memory. While conceptually similar, routing based on resident state, the physics differ fundamentally due to scale. LoRA adapters are relatively small (tens of MBs) and can be swapped quickly. In contrast, RAG workloads generate transient token-level KV caches that grow to gigabytes per request, especially for large models. Furthermore, the cost of a cache miss here is not just data loading, but the massive computational penalty of re-executing the prefill phase, requiring a fundamentally different cost model.

3.5 The Need for Lightweight vLLM Routing

Despite the innovations in routing, vLLM (Kwon et al., 2023) remains the standard for high-performance inference. Its primary contribution, PagedAttention, solves the memory fragmentation problem by managing the KV cache as non-contiguous blocks, analogous to virtual memory paging in operating systems. Furthermore, vLLM utilizes continuous batching (Yu et al., 2022) to maximize throughput by dynamically inserting new requests as soon as resources become available.

However, vLLM’s scheduler is locally optimized and currently lacks a native, lightweight routing layer that is aware of these internal memory mechanics in a distributed setting. This creates a critical gap for a plug-and-play routing logic that can be layered over standard vLLM workers, treating them as black boxes while intelligently managing the trade-off between queueing delay and prefill recomputation.

4 SYSTEM OVERVIEW

Our system comprises a centralized Control Plane (Router) and a Data Plane (Workers). The Router acts as the gateway, maintaining an eventually consistent view of the cluster’s cache state via a global radix trie (Figure 1).

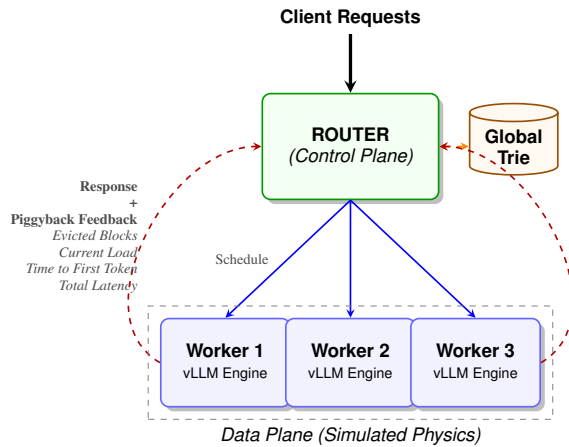


Figure 1: System Architecture. The Router distributes requests based on a cost model derived from a global prefix trie. Workers embed cache eviction events and queue statistics in HTTP response headers (piggybacking), allowing the Router to lazily update its global view.

The router utilizes a piggyback protocol, through the responses, for state management. It optimistically updates its internal state when routing a request (e.g. load, cached blocks, etc.) and corrects using meta-data (e.g., Evicted-Blocks) attached to worker responses.

5 METHODOLOGY

5.1 System Architecture and Implementation

Due to hardware constraints preventing a physical multi-node deployment, we designed a high-fidelity simulator to evaluate routing strategies. The system is implemented in Python 3.10, utilizing FastAPI and Uvicorn for the web server and Asyncio for non-blocking concurrency. To ensure our simulation serves as a realistic proxy for production environments, we explicitly mirrored the core internal mechanisms of vLLM (Kwon et al., 2023), specifically Paged Attention, continuous batching, chunked prefills, and safety-aware eviction.

- **Tokenization & Semantics:** The Llama-2-13B model from Hugging Face was utilized as the model being simulated. However, a lightweight GPT-2 model is used in reality to generate valid semantic responses and determine the output sequence length (N_{decode}). This ensures the simulator produces valid token streams while the timing is calculated separately by our physics engine.
- **Paged Attention & Memory Management:** We implemented a custom block manager mirroring vLLM’s PagedAttention. The KV cache is managed as non-contiguous fixed-size blocks ($B = 16$ tokens). Each worker maintains a Radix Trie to map token prefixes to cached blocks. To ensure correctness under high load, the memory manager implements a "Block Pinning" mechanism; blocks required by active requests are pinned before eviction occurs.
- **Eviction Policy:** When the cache is full, the eviction process selects unpinned blocks to free memory. The policy prioritizes the Least Recently Used (LRU) blocks (inactive blocks). To simplify the modification of the Radix Trie, ties are broken by the block’s sequence number, ensuring that leaf nodes are evicted before internal nodes.
- **Scheduler:** A continuous batching scheduler operates with a fixed token budget per step. It strictly prioritizes decode phase requests to minimize inter-token latency. Adopting the strategy from Sarathi Serve (Agrawal et al., 2024), following vLLM, we implement chunked prefills, allowing large prefill requests to be processed in segments to prevent stalling quick decode requests.

5.2 Physics-Based Latency Modeling

The simulator decouples logical execution from physical timing. A global `sim_clock` tracks the system state, while operation durations are derived from a roofline model calibrated to an NVIDIA A100-40GB GPU. To account for

real-world inefficiencies (e.g., kernel launch overhead, non-ideal tensor core utilization), theoretical peaks are scaled by an efficiency factor based on calibration data ($\alpha \approx 0.6$).

For any given step, the execution time, T_{step} , is determined by the bottleneck resource:

$$T_{step} = \max \left(\frac{N_{ops}}{\alpha \cdot FLOPS_{peak}}, \frac{N_{bytes}}{BW_{peak}} \right)$$

The computational load is driven by the feed-forward layers and the attention mechanism. We calculate the "delta" computation required, skipping cached prefixes where possible. Specifically, N_{new} represents only the suffix of the prompt that was not found in the cache ($N_{new} = N_{input} - N_{cached}$). In the following formulations, we denote the number of layers as L and the model hidden dimension as D .

- **Linear Layers:** This cost scales linearly with the model parameters P and the number of new tokens N_{new} .

$$N_{ops}^{linear} \approx P \cdot N_{new}$$

This formulation demonstrates the benefit of cache hits. As $N_{cached} \rightarrow N_{input}$, $N_{new} \rightarrow 0$, effectively eliminating the compute cost of the prefill phase.

- **Attention Mechanism:** Even if tokens are cached, new tokens must still attend to the existing history. The complexity depends on the interaction between new tokens (1 when decoding) and the full sequence length L_{seq} :

$$N_{ops}^{attn} \approx L \cdot D \cdot N_{new} \cdot L_{seq}$$

The memory cost, N_{bytes} , is the sum of loading static model weights and the dynamic IO of the KV cache.

- **Model Weights (W_{model}):** The entire model is loaded per forward pass, proportional to P .
- **KV Cache (W_{KV}):** During decoding, the system is bandwidth-bound as the KV history must be loaded.

$$W_{KV} \approx \sum_{req \in Batch} \left(L \cdot D \cdot L_{seq}^{(req)} \right)$$

This captures the memory wall during decoding. While N_{ops} is low, W_{KV} grows with context length, keeping the arithmetic intensity (N_{ops}/N_{bytes}) low and saturating BW_{mem} .

5.3 Calibration

To validate the simulator, we compared its latency predictions against a single vLLM instance with prefix caching running Llama-2-13B on an actual A100 GPU. We utilized

SQuAD v2 (Stanford Question Answering Dataset) for validation (Rajpurkar et al., 2018), as it models a RAG workload, which often has shared prefix. This contains pairs of questions and documents, which may repeat, that vary realistically in prompt and completion lengths. Initial runs showed a systematic under-estimation of latency. To correct this, we calculated the distribution of the error (Real - Simulator) and applied a constant bias correction offset to the simulator’s output such that the median error became zero. As shown in Figure 2, the calibrated simulator (Scenario B) achieves a highly accurate alignment with physical hardware (Mean Error ≈ 0.1 ms).

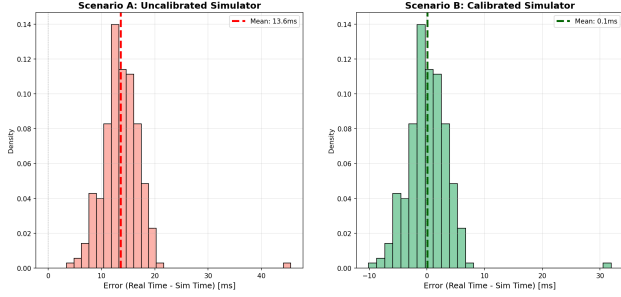


Figure 2: We applied a bias correction to align the simulator’s median latency with real A100 hardware. Calibrated results showcase the simulator being misaligned with physical hardware by 0.1ms on average.

5.4 State Management and Feedback

The router maintains an optimistic global view of the cluster state to minimize synchronization overhead.

- **Optimistic Updates & Hashing:** Upon routing a request, the router immediately updates its shadow Radix Trie. To maintain a lightweight and scalable footprint, this trie is built using hashes of the blocks rather than storing raw token data. Furthermore, the router optimistically assumes that both the prompt blocks and the eventually generated response blocks will be cached by the worker.
- **Piggyback Feedback:** Workers embed state metadata in standard HTTP responses. Headers include Evicted-Blocks, Current-Load, TTFT, and Total-Latency. When a piggyback response arrives, the router corrects its internal state to match the true state.

5.5 Routing Algorithms and Cost Model

We implemented four routing strategies to evaluate the trade-off between locality and load balancing:

1. **Round Robin:** Baseline strategy $w/i = (i + 1)\%N$.

2. **Least Loaded:** Routes to the worker with the minimum active requests.
3. **Cache Aware (Greedy):** Routes to the worker with the maximum number of matching prefix blocks.
4. **Advanced Cache Aware (Cost Model):** Explicitly prices the trade-off between queuing delay and prefill recomputation for each worker w through a score S_w :

$$S_w = T_{queue_wait}(w) + T_{prefill}(req, cache_w)$$

We estimate the wait time based on the worker’s current load and the batch size B , normalized by the generation limit N_{gen} :

$$T_{queue_wait} = \left(\frac{Load_w}{B} \right) \times (N_{gen} \times T_{decode})$$

Using the Roofline model, we calculate the cost to process only the non-cached tokens:

$$T_{prefill} = \text{Roofline}(N_{input} - N_{cached})$$

The per-token decode time is learned online via a rolling window of the last 50 requests to adapt to cluster contention:

$$T_{decode} = \frac{TotalLatency - TTFT}{N_{tokens} - 1}$$

The code for this setup can be found at <https://github.com/AmanPandey28/stateful-cache-router>.

6 EVALUATION

To validate our routing policies, we conducted experiments using the calibrated simulator described in Section 5.

6.1 Experimental Setup

Hyperparameters: Our simulator’s physics engine is calibrated to the roofline performance of an NVIDIA A100-40GB GPU. We utilized the specific hardware hyperparameters of 312 TFLOPS (peak FP16 throughput) and 1555 GB/s (memory bandwidth). For workload execution, we simulated the architectural parameters of the Llama-2-13B model ($L = 40$ layers, $d_{model} = 5120$, $N_{heads} = 40$). To account for runtime overheads not captured by the roofline model, we applied a constant calibration bias determined empirically during validation.

Datasets & Workloads: We evaluated the system using two content distributions:

- **RAG (SQuAD v2):** The same high-locality dataset used for calibration, where multiple queries share identical context passages.

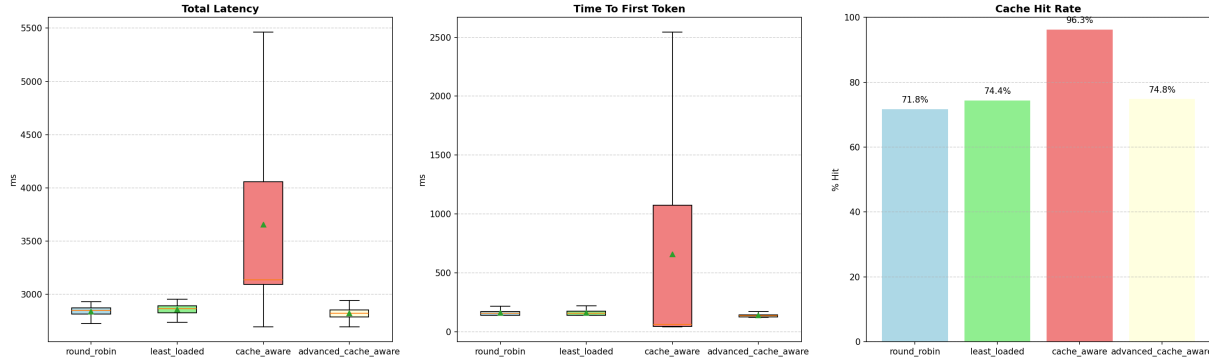


Figure 3: **SQuAD Burst.** The Greedy Cache-Aware strategy achieves a cache hit rate of approximately 95% but suffers from severe congestion, resulting in a median TTFT exceeding 0.7 s and a P99 Total Latency above 5 s. In contrast, the Advanced Cost-Aware strategy limits queue growth by routing to idle workers once saturation occurs. This reduces median TTFT to below 300 ms and lowers tail latency by more than an order of magnitude while retaining high cache reuse.

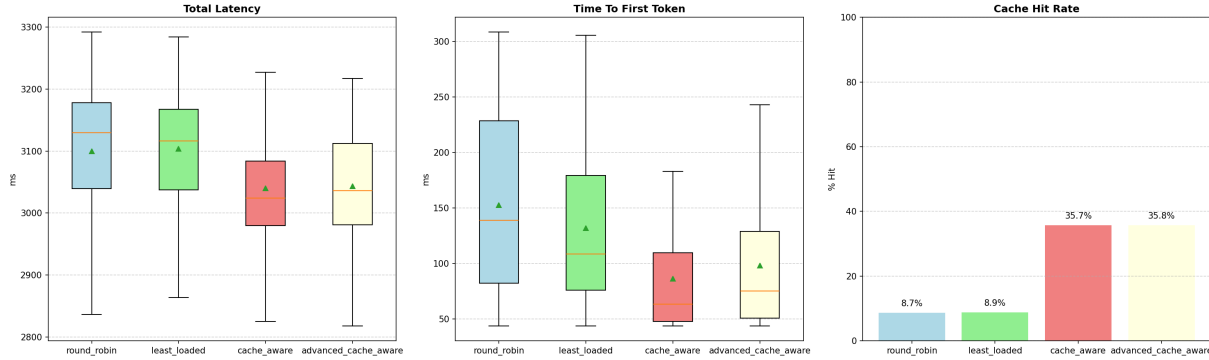


Figure 4: **Mixed Burst.** With reduced prefix locality, cache hit opportunities are limited. Cache-blind baselines (Round Robin and Least Loaded) achieve only a $\approx 9\%$ cache hit rate, while cache-aware routing increases this to $\approx 36\%$. Despite the lower locality, cache-aware routing significantly improves Time-To-First-Token: the Greedy strategy reduces median TTFT to 60–70 ms (vs. 110–140 ms for baselines). The Advanced strategy achieves similar TTFT reductions while maintaining stable end-to-end latency, preserving prefix reuse benefits without introducing queue-induced latency spikes.

- **Mixed:** A 50/50 split between SQuAD v2 samples and random noise prompts to evaluate robustness.

These datasets were tested under two arrival patterns:

- **Burst:** Simultaneous request arrival to stress queue management and expose hotspotting.
- **Streaming:** Requests arrive with an average rate below the cluster’s saturation point, simulating steady-state usage.

Metrics: We report *Time-To-First-Token (TTFT)* and *Total Latency* to measure user-perceived performance. Additionally, we track the *Cache Hit Ratio* to quantify the efficiency of our routing logic.

6.2 Results

We first address the question of whether maximizing cache hits blindly introduces resource contention. Figure 3 presents the results for the SQuAD Burst scenario.

Herding Behavior: As hypothesized, the Greedy Cache-Aware strategy maximizes the Cache Hit Ratio but suffers from severe tail latency degradation. Because specific context documents in SQuAD are popular, the greedy router “herds” all matching requests to the single worker holding that prefix. This confirms that without load awareness, high cache locality can paradoxically degrade performance.

Cost-Aware Mitigation: The Advanced Cost-Aware strategy successfully prevents this hotspotting. By pricing the wait time (T_{queue_wait}) against the prefill savings, it dynamically determines when recomputing the prefix on an idle worker is faster than waiting for the cached worker.

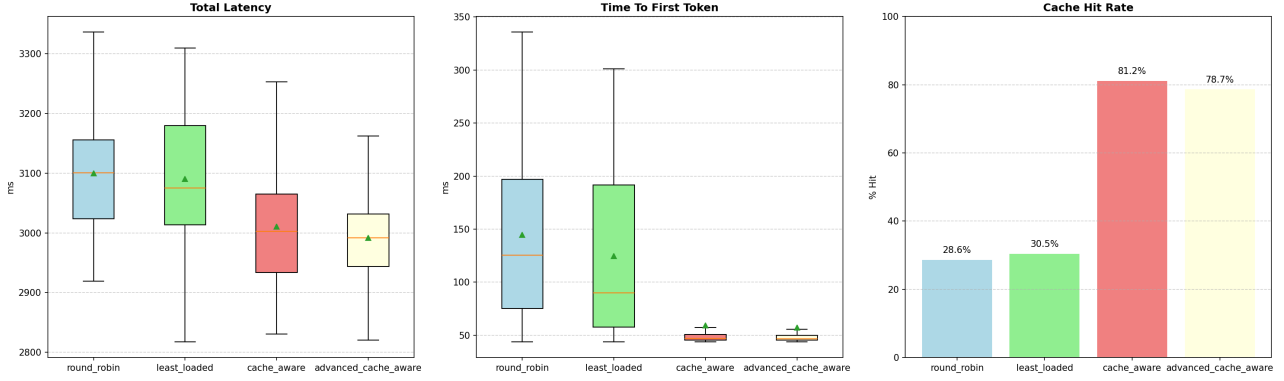


Figure 5: **SQuAD Streaming.** Under staggered arrivals, queue pressure is low, and performance is dominated by prefix reuse. Cache-blind baselines achieve moderate cache hit rates (28.6% for Round Robin, 30.5% for Least Loaded), while cache-aware routing substantially increases reuse (81.2% for Greedy, 78.7% for Advanced). This increase translates directly into improved Time-To-First-Token: median TTFT drops from 90–130 ms (baselines) to 50–55 ms (cache-aware). Total end-to-end latency remains comparable across all policies, indicating that the Advanced strategy converges toward cache-maximizing behavior in steady state without adding overhead.

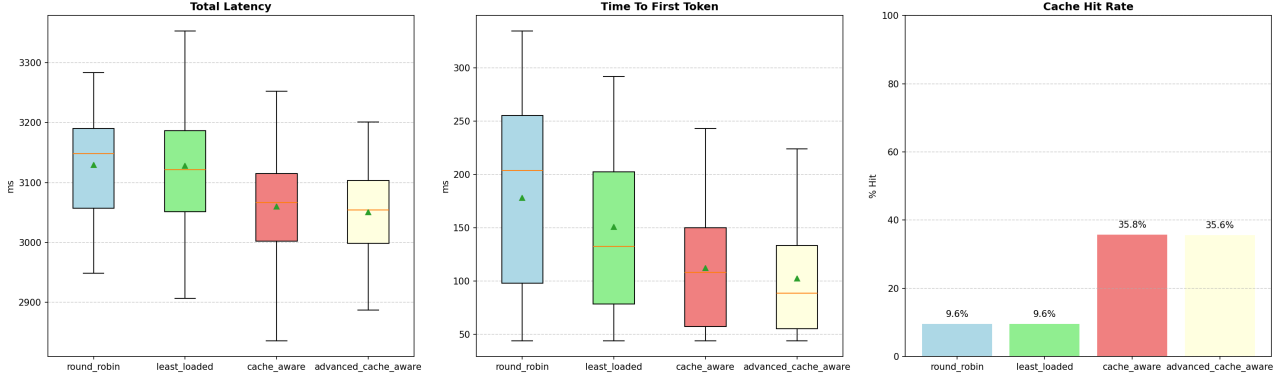


Figure 6: **Mixed Streaming.** In a streaming setting with limited prefix reuse, cache hit rates remain low for cache-blind baselines ($\approx 9.6\%$). Cache-aware routing increases the hit rate to $\approx 36\%$, though overall locality remains constrained. Despite limited reuse, cache-aware strategies reduce Time-To-First-Token: median TTFT decreases from roughly 200 ms (Round Robin) and 130 ms (Least Loaded) to 100 ms for Greedy and 85–90 ms for the Advanced strategy. Total latency remains similar across policies, confirming that the Advanced strategy improves TTFT without introducing queuing overhead in low-locality workloads.

Figure 3 demonstrates this improved performance: the system achieves significantly lower TTFT and total latency by accepting a slight trade-off in the cache hit rate.

Robustness: Figure 4 illustrates the “Mixed Burst” workload. Even with 50% random noise, the Advanced strategy maintains lower Total Latency than standard Load Balancing, demonstrating its robustness to realistic traffic.

Next, we examine whether a single policy can adapt to varying traffic patterns without manual tuning.

Convergence: In the Streaming SQuAD experiment (Figure 5), request arrivals are staggered, ensuring that queue depths rarely grow large. Under these conditions, the $T_{queue.wait}$

term in our cost model approaches zero. Consequently, the Advanced policy converges to the behavior of the Greedy policy, evidenced by the nearly identical total latency, TTFT, and cache hit rate distributions.

Steady-State Efficiency: Figure 6 shows that in mixed streaming scenarios, the Advanced strategy minimizes the aggregate computational load (N_{ops}) better than cache-blind strategies. By routing cache-hit requests to affinity workers and load-balancing random requests, it preserves the cluster’s compute capacity for actual generation rather than redundant prefill.

Through these experiments, it is clear the Advanced Cache

Aware performs the best across various traffic scenarios. It does so by weighing the queue wait time for a cache-hit worker against recomputing pre-fill on a idle worker, essentially trading off cache-hits and load balancing.

7 CONCLUSION

In this project, we presented a stateful cache-aware routing approach for distributed LLM inference. Our goal was to move beyond cache-blind load balancing by making the routing layer aware of KV cache reuse, while still remaining lightweight and practical to deploy. To do this, we combined cache awareness with a simple physics-based cost model that estimates both prefill recomputation and queueing delay. Through our evaluation, we found that aggressively maximizing cache hits is not always the right strategy. Under bursty traffic, greedy cache-aware routing can lead to request herding, where many requests queue behind a single worker, increasing tail latency. The Advanced Cache-Aware policy avoids this behavior by explicitly trading off cache reuse against queue delay. As a result, it achieves more stable Time-To-First-Token and end-to-end latency across a range of workloads, while still retaining most of the benefits of KV cache reuse.

8 FUTURE WORK

This work opens several avenues for further optimization in distributed inference systems:

- **Disaggregated Architectures:** We plan to extend our routing logic to support physically disaggregated prefill and decode pools (Zhong et al., 2024), investigating hierarchical routing topologies that coordinate state transfer between distinct hardware tiers.
- **Online Simulation-Based Routing:** We aim to integrate the event-driven simulator directly into the control plane. Utilizing the simulator as a real-time predictive oracle would allow the router to anticipate complex queuing dynamics more accurately than static heuristics.
- **Multivariate Cost Functions:** Future iterations of the Advanced policy will incorporate granular signals beyond queue length, such as exact prefix match counts and the specific prefill-to-decode ratio of pending requests, to refine the estimated cost of recomputation.
- **Heterogeneous Cluster Support:** Real-world deployments often mix accelerator generations (e.g., A100s and H100s). We intend to adapt the cost model to weight worker capacity by compute capability, ensuring latency-critical requests are biased toward higher-performance nodes.

REFERENCES

- Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B., Tumanov, A., and Ramjee, R. Taming Throughput-Latency tradeoff in LLM inference with Sarathi-Serve. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pp. 117–134, 2024.
- Chen, L. et al. Punica: Multi-tenant lora serving. *arXiv preprint arXiv:2310.18547*, 2023.
- Cheng, Y., Liu, Y., Yao, J., An, Y., Chen, X., Feng, S., Huang, Y., Shen, S., Du, K., and Jiang, J. Lmcache: An efficient kv cache layer for enterprise-scale llm inference, 2025. URL <https://arxiv.org/abs/2510.09665>.
- Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 2022.
- Gao, L., Jiang, C., Zarch, H. E., Wong, D., and Annavaram, M. Duetserve: Harmonizing prefill and decode for llm serving via adaptive gpu multiplexing, 2025. URL <https://arxiv.org/abs/2511.04791>.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Stoica, I., and Zhang, H. vLLM: Easy, fast, and cheap LLM serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023.
- llm-d Contributors, T. llm-d: A kubernetes-native high-performance distributed llm inference framework. <https://llm-d.ai/docs/architecture>, 2025. Accessed: 2025-12-14.
- Mitzenmacher, M. The power of two choices in randomized load balancing. *IEEE Transactions on Parallel and Distributed Systems*, 12(10), 2001.
- Patel, P., Choukse, E., Zhang, C., Shah, A., Neves, I., Falsafi, B., Kaufmann, A., and Anderson, T. Splitwise: Efficient generative LLM inference using phase splitting. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, pp. 1206–1219. IEEE, 2024.
- Qin, X. et al. Mooncake: A kvcache-centric disaggregated architecture for llm serving. *arXiv preprint arXiv:2311.XXXXX*, 2024.
- Rajpurkar, P., Jia, R., and Liang, P. Know what you don’t know: Unanswerable questions for squad, 2018. URL <https://arxiv.org/abs/1806.03822>.

Sheng, Y., Cao, S., Li, D., Hooper, C., Lee, N., Yang, S., Chou, C., Zhu, B., Zheng, L., Keutzer, K., et al. S-LoRA: Serving thousands of concurrent lora adapters. *arXiv preprint arXiv:2311.03285*, 2023.

Shi, X., Cai, C., Du, J., and Jia, Z. Nexus: proactive intra-gpu disaggregation of prefill and decode in llm serving, 2025. URL <https://arxiv.org/abs/2507.06608>.

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., and Chun, B.-G. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. Sglang: Efficient execution of structured language model programs, 2024. URL <https://arxiv.org/abs/2312.07104>.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving, 2024. URL <https://arxiv.org/abs/2401.09670>.